

Experiment No. - 4

Student Name: Navjot Kaur

UID: 25MCA20078

Branch: MCA

Section/Group: 1/A

Semester: 3rd

Date of Performance: 08-09-2026

Subject Name: Backend Technologies

Subject Code: 25CAP-652

Aim/Overview of the practical: Create the University Application to Represent Course Structure to the Students.

Objective:

- Understand the basics of Node.js application development.
- Create and manage JavaScript objects and arrays.
- Represent hierarchical data structures.
- Display course and subject information dynamically.
- Execute Node.js applications through the terminal.

Code for experiment:

Server.js:

```
const express = require("express");

const mongoose = require("mongoose");

const app = express();

app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/studentDB").then(() => console.log("MongoDB Connected Successfully")).catch((error) => console.log(error));

const studentSchema = new mongoose.Schema({

  name: String,

  age: Number,

  course: String

});
```

```
const Student = mongoose.model("Student", studentSchema);

const courseSchema = new mongoose.Schema({

  courseName: String,

  duration: String,

  semesters: [

    {

      semesterName: String,

      subjects: [String]

    }

  ]

});
```

```
const Course = mongoose.model(
  "Course",
  courseSchema
);
app.post("/courses", async (req, res) => {
  try {
    const course = new Course(req.body);
    await course.save();
    res.status(201).json({
      message: "Course added successfully",
      data: course
    });
  } catch (error) {
    res.status(500).json({
      message: error.message
    });
  }
});
app.get("/courses", async (req, res) => {
  try {
    const courses = await Course.find();
    res.status(200).json(courses);
  } catch (error) {
    res.status(500).json({
      message: error.message
    });
  }
});
```

```

  }
});
app.get("/courses/:id", async (req, res) => {
  try {
    const course = await Course.findById(
      req.params.id
    );
    if (!course) {
      return res.status(404).json({
        message: "Course not found"
      });
    }
    res.status(200).json(course);
  } catch (error) {
    res.status(500).json({
      message: error.message
    });
  }
});
const PORT = 3000;
app.listen(PORT, () => {
  console.log(
    `Server running on port ${PORT}`
  );
});
```

Index.html:



```
<nav>

  <div class="logo">

    UniPortal

  </div>

  <div class="nav-links">

    <a href="#home">Home</a>

    <a href="#courses">Courses</a>

    <a href="#about">About</a>

  </div>

</nav>

<section class="stats">

  <div class="stat-card">

    <div class="icon"></div>

    <h2 id="courseCount">0</h2>

    <p>Available Courses</p>

  </div>

  <div class="stat-card">

    <div class="icon"></div>

    <h2 id="semesterCount">0</h2>
```

```
<p>Total Semesters</p>

</div>

<div class="stat-card">

  <div class="icon"></div>

  <h2 id="subjectCount">0</h2>

  <p>Total Subjects</p>

</div>

</section>

<!-- Courses Section -->

<section class="courses-section"
id="courses">

  <div class="section-heading">

    <span>ACADEMIC
    PROGRAMS</span>

    <h2>Available Courses</h2>

    <p>

      Select a course to explore its
      semester-wise
      subject structure.

    </p>

  </div>
```

Script.js

```
sync function loadCourses() {

  try {

    const response = await fetch("/courses");

    const courses = await response.json();

    displayCourses(courses);

    updateStatistics(courses);

  } catch (error) {

    console.log("Error loading courses:", error);

    document.getElementById("courseContainer")
      .innerHTML = `

      <div class="loading">
```

Unable to load courses.

</div>

}}

function displayCourses(courses) {

const container =

document.getElementById("courseContainer
");

container.innerHTML = "";

if (courses.length === 0) {

container.innerHTML = `

<div class="loading">

No courses found.

</div>

return; }

courses.forEach(course => {

let semestersHTML = "";

course.semesters.forEach(semester => {

let subjectsHTML = "";

semester.subjects.forEach(subject => {

subjectsHTML += `

\${subject}

});

semestersHTML += `

<div class="semester">

<h3>

\${semester.semesterName}

</h3>

\${subjectsHTML}

</div>

});

container.innerHTML += `

<div class="course-card">

<h2 class="course-title">

\${course.courseName}

</h2>

<p class="duration">

Duration: \${course.duration}

</p>

\${semestersHTML}

</div>

});

}

function updateStatistics(courses) {

let semesterCount = 0;

let subjectCount = 0;

courses.forEach(course => {

semesterCount += course.semesters.length;

course.semesters.forEach(semester => {

subjectCount += semester.subjects.length;

});

});

document.getElementById("courseCount").inne
rText =

courses.length;

document.getElementById("semesterCount").in
nerText =

```

semesterCount;

document.getElementById("subjectCount").innerText =

    subjectCount;
}

document

.getElementById("searchInput")

.addEventListener("input", function () {

    const searchText =

        this.value.toLowerCase();

    const cards =

        document.querySelectorAll(".course-
card");

    cards.forEach(card => {

        const courseName =

```

```

card

.querySelector(".course-title")

.innerText

.toLowerCase();

if (courseName.includes(searchText)) {

    card.style.display = "block";

} else {

    card.style.display = "none";

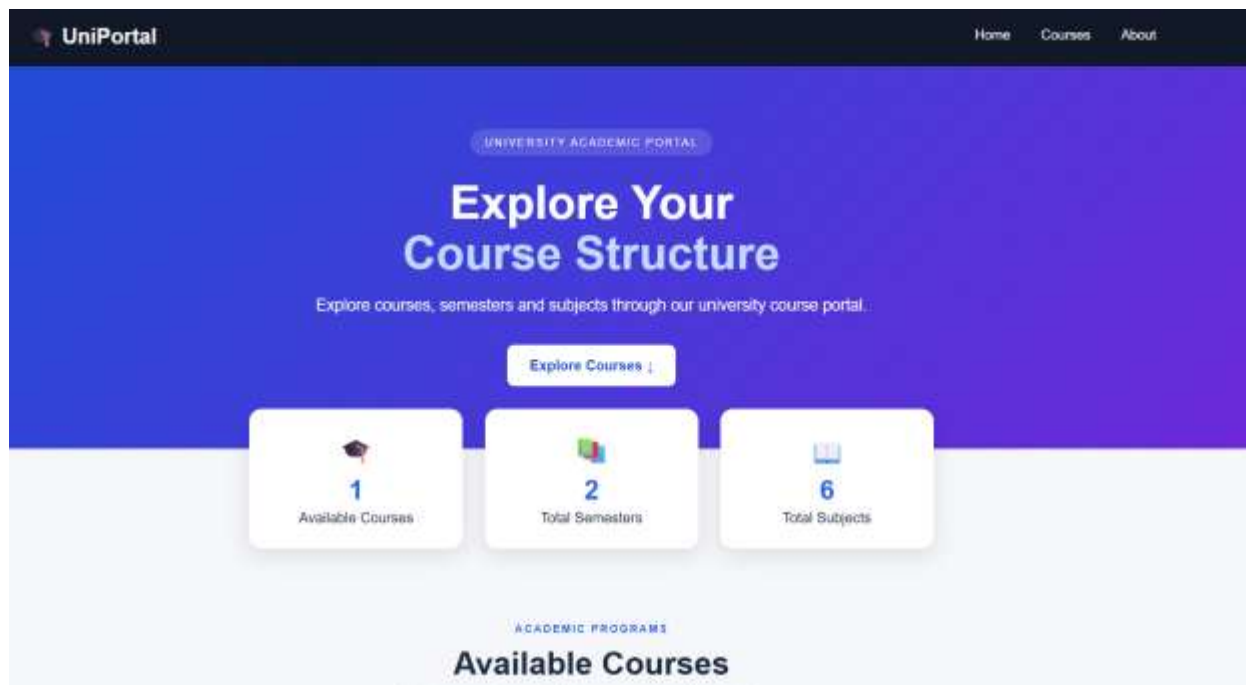
}

});

});

loadCourses();

```



ACADEMIC PROGRAMS

Available Courses

Select a course to explore its semester-wise subject structure.

Master of Computer Applications

🕒 Duration: 2 Years

Semester 1

- Data Structures
- Database Management System
- Web Development

Semester 2

- Machine Learning
- Operating System
- Computer Networks

Learning Outcomes (What I Have Learnt)

- Understand Node.js and Express application development.
- Represent course and semester structure using MongoDB.
- Retrieve and display course data dynamically.
- Develop a simple and interactive course portal.